Reinforcement Learning

Lecture 2 — Finite Markov Decision Processes

Przemysław Witaszczyk

Faculty of Physics, Astronomy and Applied Computer Science
Jagiellonian University

2026/2027

Where we are

Lecture 1 treated the multi-armed bandit as "0-dimensional" RL: no state, no sequencing.

Lecture 2 adds the missing dimension — a **state** that the agent's actions change — and the *sequential, delayed* consequences this creates. Every bandit object is *lifted*:

Bandit (L1)	$\longrightarrow$	**MDP (L2)**
reward distribution ν_a	$\longrightarrow$	dynamics $p(s', r \mid s, a)$
value $q_*(a) = \mathbb{E}[R \mid A{=}a]$	$\longrightarrow$	$v_\pi(s),\ q_\pi(s, a)$
single reward R_t	$\longrightarrow$	return $G_t = \sum_k \gamma^k R_{t+k+1}$
greedy horizon	$\longrightarrow$	episodic / continuing tasks
pick $\arg\max_a q_*(a)$	$\longrightarrow$	Bellman optimality $\rightarrow \pi_*$

The **tower property** from L1 (gradient-bandit derivation) returns as the key tool: it is what turns the return recursion into a **Bellman equation**.

The MDP tuple

and action spaces, and I'll provide you with the discount factors. We'll assume horizons of infinity, and won't need the initial state distribution. Additionally, the most crucial part of this chapter is to give you an awareness of the components of MDPs and POMDPs. Remember, you won't have to do much more building of MDPs than what you've done in this chapter. Nevertheless, let me define MDPs and POMDPs so we're in sync.



SHOW ME THE MATH

MDPs vs. POMDPs

$$MDP(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{S}_\theta, \gamma, \mathcal{H})$$

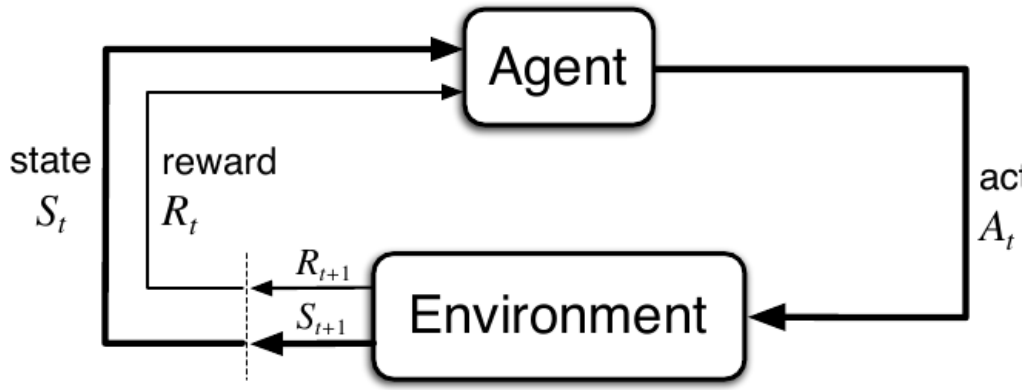
(1) MDPs have state space S, action space A, transition function T, reward signal R. They also has a set of initial states distribution S_θ, the discount factor γ, and the horizon H.

$$POMDP(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{S}_\theta, \gamma, \mathcal{H}, \mathcal{O}, \mathcal{E})$$

(2) To define a POMDP, you add the observation space O and an emission probability E that defines the probability of showing an observation o given a state s. Very simple.

Lecture 1 stated this tuple; its core is built here, together with the equations that characterize its solution. Morales' tuple and Sutton's MDP, specified by p, are the *same* object: the transition function T and reward function R fold into the joint dynamics p; the start distribution S_θ and horizon H Sutton leaves implicit (episodic tasks carry H via terminal states). (Morales, Ch. 2)

The interface and the trajectory



e 3.1: The agent–environment interaction in a Markov decis

At each step t: the agent observes state $S_t \in \mathcal{S}$, takes action $A_t \in \mathcal{A}(S_t)$, and receives reward $R_{t+1} \in \mathbb{R}$ and next state S_{t+1}. This produces a **trajectory**

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$$

Indexing convention: the reward for action A_t is R_{t+1} — it is "jointly determined" with S_{t+1}. (Both conventions exist in the literature; we follow Sutton & Barto.)

The dynamics function p

A *finite* MDP is fully specified by its **dynamics**: a four-argument function ("$\doteq$" means "is defined as")

$$p(s', r \mid s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}, \quad (3.2)$$

defined for all $s', s \in \mathcal{S}$, $r \in \mathcal{R}$, $a \in \mathcal{A}(s)$. It is a proper conditional distribution:

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1, \qquad \text{for all } s \in \mathcal{S},\ a \in \mathcal{A}(s). \quad (3.3)$$

- *Finite* means finite throughout: $\mathcal{S}$, $\mathcal{A}$, and the set of possible rewards $\mathcal{R} \subset \mathbb{R}$ all have finitely many elements — which is what makes every $\sum_{s',r}$ here a legitimate finite sum.
- p **completely characterizes** the environment: everything else is derived from it.
- This generalizes the bandit's per-arm reward law ν_a — now the *state* can change too.

Quantities derived from p

State-transition probabilities (marginalize over reward):

$$p(s' \mid s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a). \tag{3.4}$$

Expected reward for a state–action pair:

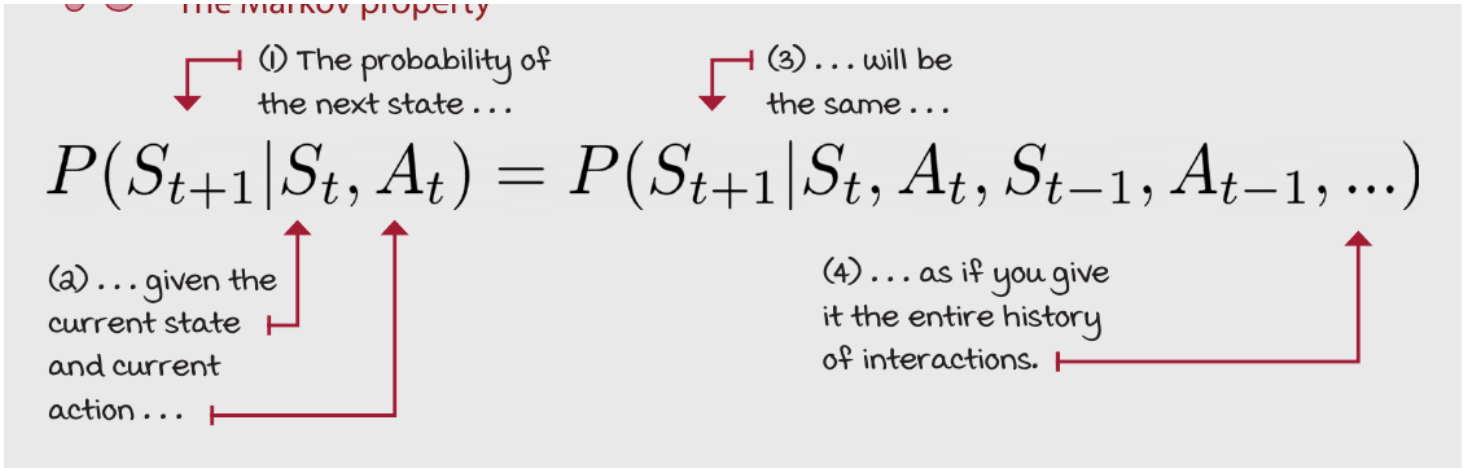
$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_r r \sum_{s'} p(s', r \mid s, a). \tag{3.5}$$

Expected reward for a state–action–next-state triple:

$$\begin{aligned} r(s, a, s') &\doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] \\ &= \sum_r r \, \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}. \end{aligned} \tag{3.6}$$

We mostly use the four-argument p; these forms are occasionally convenient. (They are Morales' transition $p(s'|s, a)$ and reward functions $r(s, a)$, $r(s, a, s')$ — named in the tuple at the close of L1.)

The Markov property



But why do you care about this? Well, in the environments we've explored so far it's not that obvious, and it's not that important. But because most RL (and DRL) agents are designed to

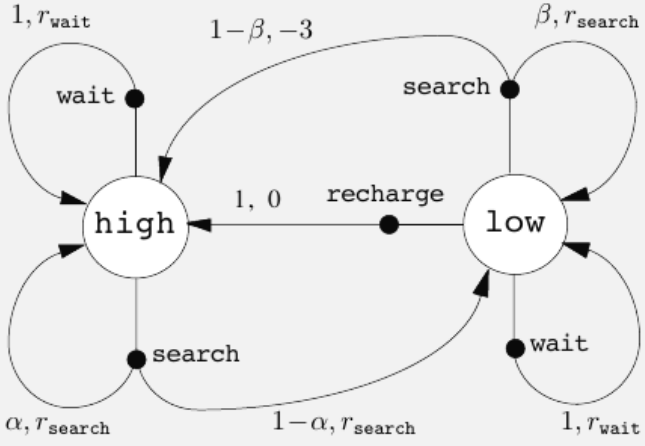
The probability of (S_t, R_t) depends on the past *only* through (S_{t-1}, A_{t-1}). This is a restriction on the **state**, not the process: the state must summarize everything about the past that matters for the future.

The property is assumed throughout the tabular half of the course; states that features cannot tell apart appear in Lecture 10, and partially observable problems (POMDPs) in Lecture 12.

Worked example: the Recycling Robot

:pected reward) while searching and while waiting respectively. Finally, suppose that no ıns can be collected during a run home for recharging, and that no cans can be collected ı a step in which the battery is depleted. This system is then a finite MDP, and we ın write down the transition probabilities and the expected rewards, with dynamics as dicated in the table on the left:

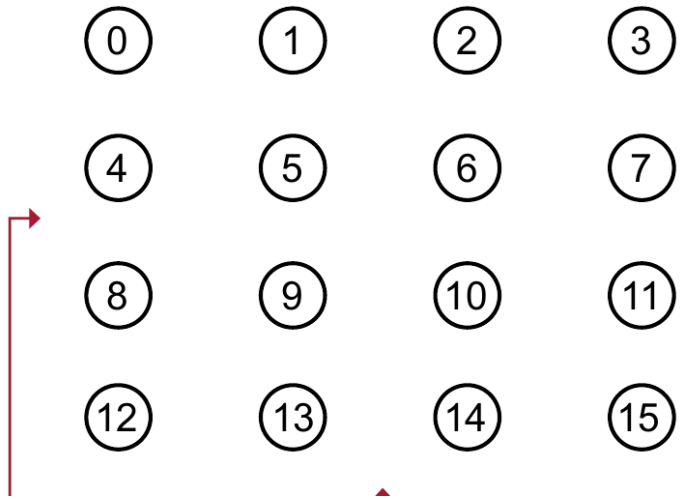
s	a	s'	$p(s'\|s,a)$	$r(s,a,s')$
high	search	high	α	$r_{\texttt{search}}$
high	search	low	$1-\alpha$	$r_{\texttt{search}}$
low	search	high	$1-\beta$	-3
low	search	low	β	$r_{\texttt{search}}$
high	wait	high	1	$r_{\texttt{wait}}$
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	$r_{\texttt{wait}}$
low	recharge	high	1	0
low	recharge	low	0	-



ote that there is a row in the table for each possible combination of current state, s, :tion, $a \in \mathcal{A}(s)$, and next state, s'. Some transitions have zero probability of occurring,

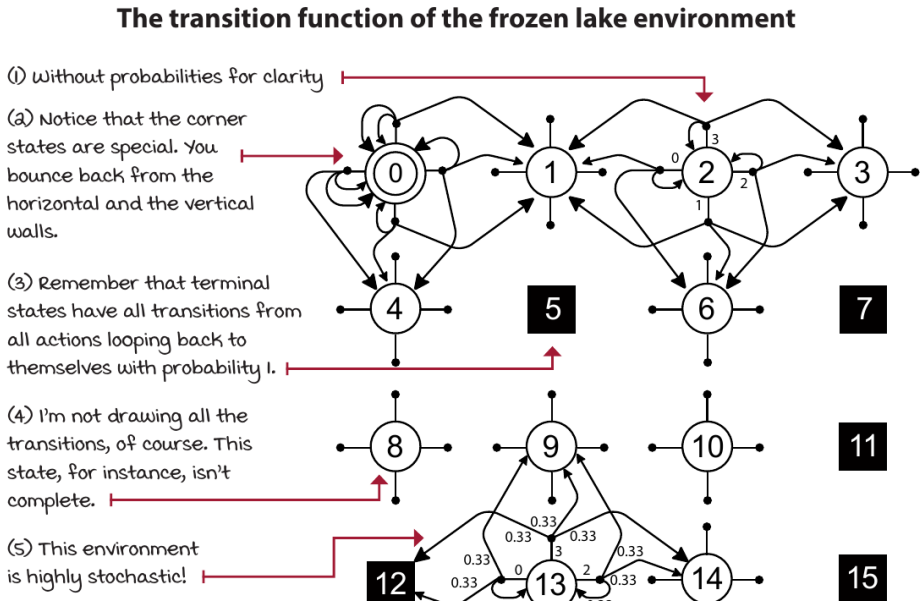
States {high,low} (battery); actions {search,wait,recharge}. The table (left) is the dynamics $p(s'|s,a)$ and expected rewards; the graph (right) is the same MDP. Searching may deplete the battery; depleting fully forces a costly rescue (-3). (Sutton & Barto, Ex. 3.3)

Worked example: Frozen Lake — our running environment



A 4×4 frozen pond: **16 states**, numbered row-wise from the `START` (state 0, top-left) to the `GOAL` (state 15, reward $+1$ on arrival). Four **holes** (states 5, 7, 11, 12) end the episode with reward 0; the goal ends it too (with the $+1$), so the task is episodic. Actions: `left`/`down`/`right`/`up`. This is the MDP we will solve exactly in Lecture 3 (DP) and learn model-free in Lecture 4. (Morales, Ch. 2)

Frozen Lake: the dynamics p, stated explicitly



The ice is **slippery**: an action moves as intended only with probability $\frac{1}{3}$, and slides to each *orthogonal* neighbour with probability $\frac{1}{3}$ (bumping the edge = staying put). In code the whole model is a lookup table: for every (s, a), a list of `(prob, next_state, reward, done)` tuples — exactly our four-argument $p(s', r \mid s, a)$, stored explicitly. "Having the model" (DP, Lecture 3) means having a table of exactly this kind.

Goals and the reward hypothesis

The reward hypothesis (Sutton & Barto, §3.2)

All of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).

- The reward $R_t \in \mathbb{R}$ encodes *what* to achieve, not *how*.
- Reward design is subtle: the reward should encode the goal (e.g. winning), not subgoals (e.g. taking the opponent's pieces) — otherwise the agent may optimize the proxy.

Principled reward design is a research topic in its own right; we return to it in Lecture 12.

Returns: episodic tasks

The agent maximizes *expected return* — a function of the reward sequence $R_{t+1}, R_{t+2}, \ldots$. In the simplest (episodic) case it is the sum until a final step T:

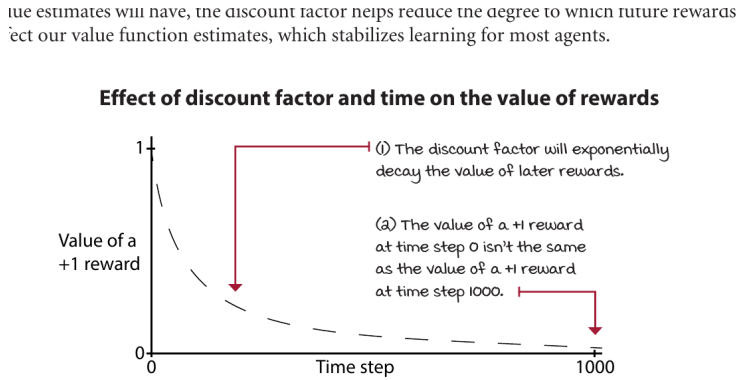
$$G_t \doteq R_{t+1} + R_{t+2} + \cdots + R_T. \tag{3.7}$$

- An **episode** ends in a **terminal state**, then resets — e.g. a game, a maze run. (Recall L1's bandits: a one-step episode.)
- T is a *random variable*, varying across episodes.
- $\mathcal{S}$ = nonterminal states; $\mathcal{S}^+$ = states plus the terminal state.

Returns: discounting and continuing tasks

For **continuing** tasks ($T = \infty$) the plain sum may diverge. Introduce a **discount rate** $\gamma \in [0, 1]$:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$
$$= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \qquad (3.8)$$



- A reward k steps in the future is worth γ^{k-1} times an immediate one.
- Finite whenever $\{R_k\}$ is bounded and $\gamma < 1$.
- $\gamma = 0$: *myopic* (only R_{t+1}). $\gamma \to 1$: *farsighted*.

Discounting is more than a convergence device. It encodes a time preference — a reward now is worth more than the same reward later — and reflects uncertainty about a far future the agent may never reach. So γ is both a mathematical convenience *and* a modelling choice.

Example: pole-balancing two ways

Keep a hinged pole upright on a moving cart; "failure" = pole falls or cart leaves the track, after which the pole is reset. The *same* task admits two formulations:

Episodic. Each balancing attempt is an episode; reward $+1$ every step without failure:

$$G_t = \sum_k 1 = (\#\text{steps until failure}).$$

Maximizing G_t maximizes the steps before failure; balancing forever gives return ∞.

Continuing. One never-ending task; reward -1 on each failure, 0 otherwise, discounted:

$$G_t = -\gamma^{K-1} - \cdots,$$

where K = steps until the next failure; the dots are later failures, each smaller. A later failure gives a larger (less negative) return.

In *both* cases the return is maximized by keeping the pole balanced as long as possible — reward design and task formulation are modelling choices, not dictated by the physical task. (Sutton & Barto, Ex. 3.4)

The recursive return identity — the basis of every Bellman equation

Returns at successive steps are related by a single identity, used in every derivation below:

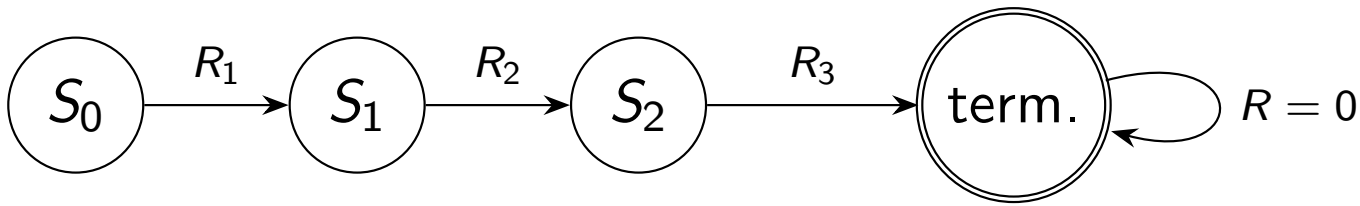
$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \\ &= R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \cdots\big) \\ &= R_{t+1} + \gamma\, G_{t+1}. \end{aligned} \tag{3.9}$$

- Every **Bellman equation** in the course is derived from this one line.
- It holds for all $t < T$; with the convention $G_T = 0$ it also covers the episodic case.

(Morales builds the same identity step by step: return $\rightarrow$ discounted return $\rightarrow$ recursive form.)

Unified notation for episodic and continuing tasks

A single formalism covering episodic and continuing tasks is preferable. Treat episode termination as entering an **absorbing state** that loops to itself with reward 0:



Then both cases are written uniformly as

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k,$$

allowing $T = \infty$ *or* $\gamma = 1$ — but *not both*, since that is the one combination that can diverge (infinitely many undiscounted rewards). Episodic tasks take a finite T (and may use $\gamma = 1$); continuing tasks take $T = \infty$ (and need $\gamma < 1$). From here on, results are stated once, in a form covering both cases.

One deferred alternative: for continuing tasks, discounting is not the only option — one can instead optimise the long-run **average reward** directly, with no γ. That needs machinery we build later; we return to it in Lecture 8.

Policies

A **policy** π maps states to distributions over actions:

$$\pi(a \mid s) \;=\; \Pr\{A_t = a \mid S_t = s\}.$$

- For each s, $\pi(\cdot \mid s)$ is a probability distribution over $\mathcal{A}(s)$.
- **Deterministic** policy: a special case, all mass on one action, written $a = \pi(s)$.
- RL is the study of methods for *changing* π from experience.

This is the same object as L1's soft-max policy $\pi_t(a)$ — now *conditioned on state*.

State- and action-value functions

Fix a policy π. The **state-value function** is the expected return from s:

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right]. \quad (3.12)$$

The **action-value function** additionally fixes the first action a:

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (3.13)$$

- These lift L1's scalar $q_*(a)$ to *functions of state*.
- The value of a terminal state is 0.
- Estimating v_π, q_π by averaging sampled returns is *Monte Carlo* (Lecture 4).

The tower property, restated from Lecture 1

Every Bellman derivation below rests on one identity from L1, restated here so that the derivations are self-contained. The **law of iterated expectation** ("tower property"): for random variables X, Y,

$$\mathbb{E}[X] = \mathbb{E}\big[\mathbb{E}[X \mid Y]\big],$$

and, for a function $g(Y)$ that is fixed once Y is known,

$$\mathbb{E}[g(Y)\,X] = \mathbb{E}\big[g(Y)\,\mathbb{E}[X \mid Y]\big].$$

How we use it here: a return G_t decomposes as $R_{t+1} + \gamma G_{t+1}$ (the recursion 3.9). Conditioning on the *next* state $S_{t+1} = s'$, the tower property lets us replace the remaining future $\mathbb{E}[G_{t+1} \mid S_{t+1} = s']$ by the value $v_\pi(s')$ — because by the Markov property, once s' is known the future does not depend on how we reached it.

That single move — "condition on the next step, then the rest of the future *is* a value function" — is what turns the return recursion into a **Bellman equation**. It recurs in every derivation that follows.

The two value functions determine each other

(i) v_π from q_π. The action $A_t \sim \pi(\cdot \mid s)$, so v_π is the π-average of q_π (tower property over the first action):

$$v_\pi(s) = \mathbb{E}_\pi[\, q_\pi(s, A_t) \mid S_t = s \,] = \sum_a \pi(a \mid s)\, q_\pi(s, a).$$

(Exercise 3.12)

(ii) q_π from v_π. Expand q_π with the return recursion (3.9), then take the tower step over the first transition $(S_{t+1}, R_{t+1}) \sim p$:

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t{=}s, A_t{=}a] \\ &= \sum_{s', r} p(s', r \mid s, a) \big[\, r + \gamma\, \mathbb{E}_\pi[G_{t+1} \mid S_{t+1}{=}s'] \,\big] \\ &= \sum_{s', r} p(s', r \mid s, a) \big[\, r + \gamma\, v_\pi(s') \,\big]. \end{aligned}$$

(Exercise 3.13)

Substituting (ii) into (i) gives the Bellman equation for v_π; substituting (i) into (ii) gives it for q_π (both derived below).

The Bellman expectation equation — derivation

Starting from the definition and the return recursion (3.9):

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma\, G_{t+1} \mid S_t = s] && \text{(by 3.9)} \\
&= \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \Big[r + \gamma\, \mathbb{E}_\pi[G_{t+1} \mid S_{t+1}{=}s'] \Big] && (\star) \\
&= \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma\, v_\pi(s') \big]. && (3.14)
\end{aligned}
$$

The key step ($\star$) is the tower property (L1): conditioning on $S_{t+1} = s'$ replaces $\mathbb{E}_\pi[G_{t+1} \mid \cdots]$ by $v_\pi(s')$, because given s' the future is independent of how we got there (Markov).

The Bellman expectation equation for q_π

Combining identities (i) and (ii) gives the recursion entirely in terms of q_π: start from (ii), then substitute (i) at the next state, $v_\pi(s') = \sum_{a'} \pi(a' \mid s')\, q_\pi(s', a')$:

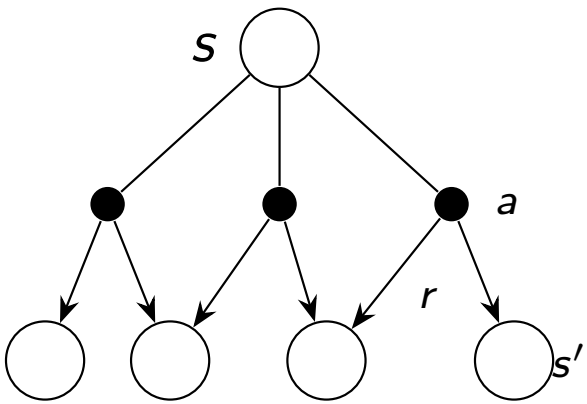
$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\big[\, r + \gamma\, v_\pi(s') \,\big] \qquad \text{(ii)}$$

$$= \sum_{s',r} p(s', r \mid s, a)\Big[\, r + \gamma \sum_{a'} \pi(a' \mid s')\, q_\pi(s', a') \,\Big]. \qquad \text{(Bellman, } q_\pi\text{)}$$

- Same structure as the v_π equation, but the inner average is over the *next action* a' instead of the current one.
- This q_π form is what control algorithms (SARSA, Lecture 5) learn — it needs no separate model to act, because q_π already ranks actions.

Reading the Bellman equation off a backup diagram

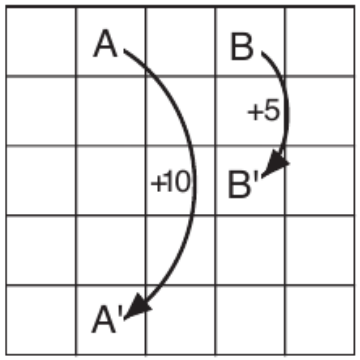
$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \, v_\pi(s') \right]$$

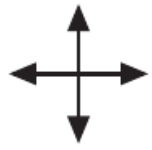


backup diagram for v_π

- Open node = state; solid node = action.
- Branch down on $a \sim \pi(\cdot \mid s)$, then on $(s', r) \sim p$.
- Each path contributes $r + \gamma v_\pi(s')$, *weighted by its probability* — the equation *is* the weighted average over the tree.
- The backup for q_π is the same tree rooted at a solid (s, a) node.

Worked example: Gridworld v_π





3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

ıre 3.2: Gridworld example: exceptional reward dynamics (left) and state-value func

5×5 grid; moving off-edge: reward -1, stay put. Special jumps: $\mathrm{A} \to \mathrm{A}'$ $(+10)$, $\mathrm{B} \to \mathrm{B}'$ $(+5)$. *Right:* v_π for the equiprobable random policy ($\gamma = 0.9$) — the *unique* solution of the linear Bellman system (3.14). A's value is below 10 because the jump ends near an edge, where later moves incur penalties.
(Sutton & Barto, Fig. 3.2)

Numerical check: Bellman holds at the center cell

The Bellman equation (3.14) must hold at *every* state. Check the center cell ($v_\pi = 0.7$). Its four interior neighbours are valued 2.3 (up), 0.4 (right), -0.4 (down), 0.7 (left); interior moves give reward 0, deterministically, and π is equiprobable ($\frac{1}{4}$ each), $\gamma = 0.9$:

$$v_\pi(s) = \sum_a \tfrac{1}{4} \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_\pi(s')\big]$$

$$= \tfrac{1}{4}(0.9)\big[2.3 + 0.4 - 0.4 + 0.7\big] = \tfrac{1}{4}(0.9)(3.0) = 0.675 \approx 0.7. \quad \checkmark$$

- The deterministic dynamics collapse $\sum_{s',r} p(\cdot)[\cdots]$ to a single term per action; the $\sum_a \frac{1}{4}$ averages the four neighbours.
- The residual is due to rounding only (values are given to one decimal). (Exercise 3.14)

Optimal policies and optimal value functions

Value functions induce a **partial order** on policies:

$$\pi \geq \pi' \iff v_\pi(s) \geq v_{\pi'}(s) \text{ for all } s \in \mathcal{S}.$$

There is always at least one **optimal policy** π_* (possibly several), all sharing the same **optimal value functions**:

$$v_*(s) \doteq \max_\pi v_\pi(s), \qquad q_*(s,a) \doteq \max_\pi q_\pi(s,a). \qquad (3.15,\ 3.16)$$

And q_* relates to v_* by one step of lookahead:

$$q_*(s,a) = \mathbb{E}\big[R_{t+1} + \gamma\, v_*(S_{t+1}) \,|\, S_t = s, A_t = a\big]. \qquad (3.17)$$

The goal of RL is an optimal policy π_* — usually reached by finding, or approximating, v_* or q_*.

The Bellman optimality equation — derivation

v_* satisfies a consistency condition with *no reference to any policy*: under an optimal policy the value of a state equals the expected return of its *best* action.

$$
\begin{aligned}
v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s,a) \\
&= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma\, G_{t+1} \mid S_t = s, A_t = a] && \text{(by 3.9)} \\
&= \max_a \mathbb{E}[R_{t+1} + \gamma\, v_*(S_{t+1}) \mid S_t = s, A_t = a] && \text{(3.18) } (\star) \\
&= \max_a \sum_{s',r} p(s', r \mid s, a)\big[\, r + \gamma\, v_*(s') \,\big]. && \text{(3.19)}
\end{aligned}
$$

$(\star)$ is the *same* tower-property move as in the expectation derivation — condition on S_{t+1} — plus one new fact: following π_* from S_{t+1} onward is worth exactly $v_*(S_{t+1})$. And for q_*:

$$
q_*(s,a) = \sum_{s',r} p(s', r \mid s, a)\Big[\, r + \gamma \max_{a'} q_*(s', a') \,\Big]. \qquad (3.20)
$$

Expectation vs. optimality: one structural change

$$\underbrace{v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(\cdot)\big[r + \gamma v_\pi(s')\big]}_{\text{Bellman } \textit{expectation}}$$

$$\underbrace{v_*(s) = \max_a \sum_{s',r} p(\cdot)\big[r + \gamma v_*(s')\big]}_{\text{Bellman } \textit{optimality}}$$

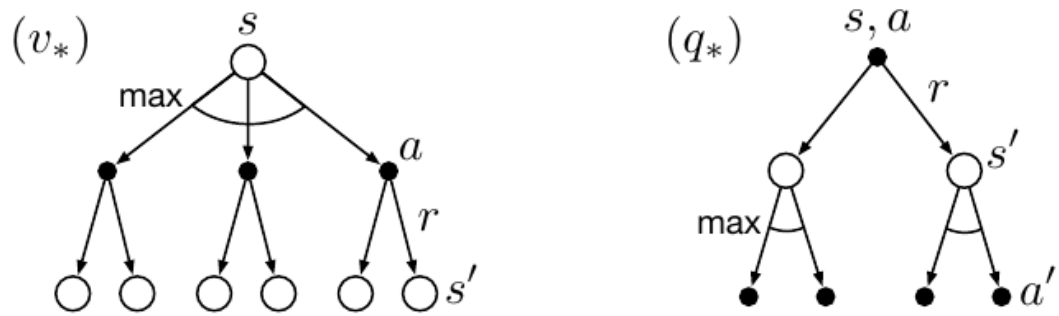


Figure 3.4: Backup diagrams for v_* and q_*

The averaging $\sum_a \pi(a \mid s)$ over actions becomes a **maximization** $\max_a$ — shown by the *arc* across the action branches. (Sutton & Barto, Fig. 3.4)

Worked example: Bellman optimality for the Recycling Robot

Instantiate (3.19) on the two-state robot ($h =$ high, $l =$ low; actions $s =$ search, $w =$ wait, $re =$ recharge). With only two states it is *two* coupled nonlinear equations. Searching from `high` keeps the battery high w.p. α:

$$v_*(h) = \max \begin{cases} r_s + \gamma\big[\alpha\, v_*(h) + (1-\alpha)\, v_*(l)\big], & \text{(search)} \\ r_w + \gamma\, v_*(h), & \text{(wait)} \end{cases}$$

$$v_*(l) = \max \begin{cases} \beta\, r_s - 3(1-\beta) + \gamma\big[(1-\beta) v_*(h) + \beta\, v_*(l)\big], & \text{(search)} \\ r_w + \gamma\, v_*(l), & \text{(wait)} \\ \gamma\, v_*(h). & \text{(recharge)} \end{cases}$$

The abstract $\max_a \sum_{s',r} p(\cdot)[r + \gamma v_*(s')]$ becomes concrete arithmetic. For any fixed $r_s, r_w, \alpha, \beta, \gamma$ ($0 \le \gamma < 1$; here α, β are Sutton's battery-transition probabilities, *not* a step size) there is *exactly one* pair $(v_*(h), v_*(l))$ solving the system — and reading off the arg max in each row yields π_*. (Sutton & Barto, Ex. 3.9)

From optimal values to an optimal policy

Once v_* (or q_*) is known, an optimal policy is *greedy* with respect to it:

- With v_*: act greedily over a **one-step lookahead** (needs the model p):

$$\pi_*(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_*(s')\big].$$

- With q_*: act greedily **directly** — *no model, no lookahead*:

$$\pi_*(s) \in \arg\max_a q_*(s, a).$$

Any policy that is greedy w.r.t. v_* is optimal — a one-step-greedy choice on v_* is globally optimal because v_* already accounts for all future rewards. This is why q_* is the more convenient target, and why model-free control (from Lecture 4 on) estimates it.

Worked example: Gridworld optimal solution

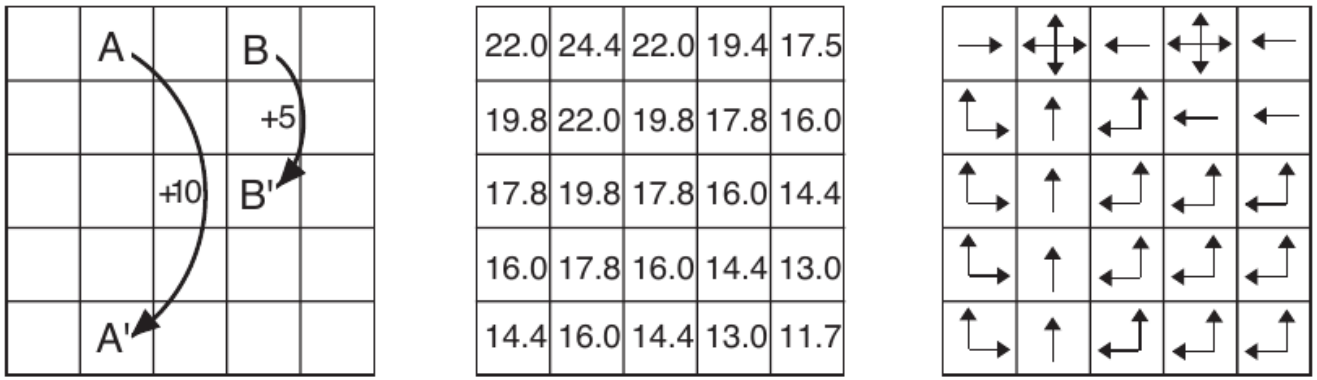


Figure 3.5: Optimal solutions to the gridworld example

Solving the Bellman optimality equation (3.19) for the same gridworld gives v_* (middle) and the optimal policies π_* (right). Multiple arrows in a cell = all those actions are optimal. Under π_* the agent moves toward A rather than following the random walk. (Sutton & Barto, Fig. 3.5)

Why exact solution is rarely available

The Bellman optimality equation is, in principle, *solvable*: for a finite MDP with n states it is a system of n nonlinear equations in n unknowns.

But an exact solution presumes three things that rarely all hold:

1. the dynamics p are **known** and accurate;
2. the **Markov** property holds;
3. there is enough **compute and memory** to solve and store it.

The **curse of dimensionality**: state spaces are typically far too large to enumerate (chess, robotics, dialogue). Backgammon has $\sim 10^{20}$ states — no table can store them. The remainder of the course therefore uses sampling (Lectures 4–7) and function approximation (Lectures 8+) to *approximate* v_*, q_*, π_*.

Most RL methods approximately solve the Bellman optimality equation

Nearly every method in this course *approximately* solves the Bellman optimality equation; the methods differ in which assumption they relax and how they treat the right-hand side $\max_a \sum_{s',r} p(\cdot)[r + \gamma v_*(s')]$:

- **Heuristic search** (e.g. A^*): expand the RHS several steps into a lookahead *tree*, then estimate v_* at the leaves with a heuristic. Relaxes (3) compute by going only to a limited depth. (Lecture 7.)
- **Dynamic programming** (Lecture 3): iterate the RHS as an *operator* over the whole state set — relaxes nothing: exact while (1)–(3) all hold.
- **Reinforcement learning** (Lectures 4+): replace the expected transition $\sum_{s',r} p(\cdot)[\cdots]$ by *actual sampled* transitions — relaxes (1), no model needed.

In this view, each later algorithm computes the Bellman right-hand side by some tractable means.

From characterization to computation

We have characterized the solution (v_*, q_*) but not *computed* it.

- The Bellman *expectation* equation (3.14) is **linear** in v_π — solvable directly, but $O(n^3)$.
- The Bellman *optimality* equation (3.19) is **nonlinear** (the max) — no closed form.

Next lecture — Dynamic Programming: these equations become *iterative operators*. Sweeping the expectation equation gives **policy evaluation**; greedifying gives **policy improvement**; interleaving them is **Generalized Policy Iteration** — the template almost every later algorithm instantiates.

Summary — Lecture 2

The finite MDP $= (\mathcal{S}, \mathcal{A}, p, r, \gamma)$, with dynamics $p(s', r \mid s, a)$ and the Markov property.

The four Bellman equations (the pattern matters, not the symbols):

$$
\begin{aligned}
v_\pi(s) &= \textstyle\sum_a \pi(a|s) \sum_{s',r} p[r + \gamma v_\pi(s')] && \text{expectation, } v_\pi \\
q_\pi(s,a) &= \textstyle\sum_{s',r} p[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')] && \text{expectation, } q_\pi \\
v_*(s) &= \max_a \textstyle\sum_{s',r} p[r + \gamma v_*(s')] && \text{optimality, } v_* \\
q_*(s,a) &= \textstyle\sum_{s',r} p[r + \gamma \max_{a'} q_*(s',a')] && \text{optimality, } q_*
\end{aligned}
$$

The recurring derivation pattern combines the return recursion (3.9), the tower property, and the expansion of the expectation as a sum over p.

Lecture 3: Dynamic Programming — solving the equations for v_π and v_* iteratively when p is known.